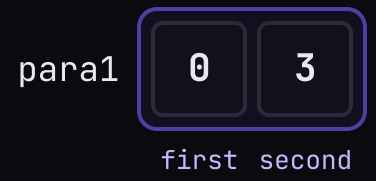
// T-OLIMPIANS — PREZENTACJA DO LEKCJI

Pary w C++ — krok po kroku

Wszystko o pair, powoli i po kolei: jak stworzyć parę, co jest w niej na starcie, jak dostać się do first i second, jak je wypisać i zmienić, jak zrobić tablicę par, tymczasową parę z make_pair i jak wczytać wiele par. Każdy krok na osobnym slajdzie — z kodem i tym, co pojawi się na ekranie.

C++ · LEKCJA 9 · PAIR · PO TABLICACH I STRINGACH



PLAN PREZENTACJI

01 Po co nam para?
02 Tworzenie pary
03 Para bez wartości
04 first i second
05 Wypisywanie pary
06 Zmiana jednego elementu
07 Zmiana obu naraz
08 Różne typy w parze
09 Tablica par
10 Tymczasowa para
11 Wczytywanie par
12 Para w parze
13 Sprawdź się
14 Najczęstsze błędy
15 Podsumowanie

01 Dwie wartości, które są razem

Często dwie liczby **należą do siebie** i nie ma sensu trzymać ich osobno:

`Punkt na kratce`
(x, y) — np. **(3, 5)**

`Data`
(dzień, miesiąc) — np. **(24, 10)**

`Wynik zawodnika`
(numer, punkty) — np. **(7, 95)**

DO TEJ PORY

```
int x = 3;    // te dwie zmienne
int y = 5;    // opisują JEDEN punkt
```

Działa, ale przy 10 punktach mamy już **20** osobnych zmiennych: x1, y1, x2, y2, … Łatwo pomylić, które x pasuje do którego y.

`Wniosek:` potrzebujemy **jednej zmiennej**, która trzyma **dwie wartości naraz**. To jest **para** — po angielsku **pair**.

01 Para = pudełko z dwiema przegródkami

Para to **jedna zmienna** z **dwiema przegródkami**. Pierwsza przegródka nazywa się **first** (pierwszy), druga — **second** (drugi).



1. Cała para ma **jedną nazwę** — tutaj `punkt`.
2. W środku są **dokładnie dwie** wartości — nie mniej i nie więcej.
3. Przegródki **nie mają numerów** jak szufladki w tablicy — mają **nazwy**: first i second.
4. Każda przegródka może mieć **inny typ**, np. litera i liczba — o tym w rozdziale 08.

01 Szablon programu z parą

SZABLON KAŻDEGO PROGRAMU

```
#include <iostream>    // cin i cout
#include <utility>     // tu mieszka pair
using namespace std;   // pair zamiast std::pair

int main(){
    // <- tu wpisujemy kod z parami
}
```

Para jest w bibliotece **utility**. Dopisujemy ją na górze, obok iostream.

> W praktyce para często działa już z samym iostream, ale pewniej jest dopisać `#include <utility>`.

Na kolejnych slajdach często pokazujemy tylko **środek main** — reszta programu jest zawsze taka sama jak tutaj.

02 Budowa linijki, która tworzy parę

Rozłóżmy na części linijkę z notatek. Każdy kawałek ma swoje zadanie:

```
pair<int, int> zmienna;
```

1 2 3 4

1	`pair`	„to będzie **para**”
2	`<int, int>`	w nawiasach ostrych: **jakie typy** przechowuje para — pierwszy typ dla first, drugi dla second
3	`zmienna`	**nazwa** — wymyślamy ją sami (np. para1, punkt, wynik)
4	`;`	**średnik** na końcu — jak po każdym poleceniu

Razem `pair<int, int>` to **typ** — tak jak `int`. Dalej jest jak przy zwykłej zmiennej: typ, nazwa, średnik.

02 Przeczytajmy to po polsku

```
pair<int, int> zmienna;
```

1. `pair` → „Zrób **parę**…”
2. `<int, int>` → „…w której pierwsza wartość to **int** i druga wartość to **int**…”
3. `zmienna` → „…i nazwij ją **zmienna**.”



Nie podaliśmy żadnych wartości. Co jest wtedy w środku? To temat rozdziału 03.

02 Tworzymy parę od razu z wartościami

Z NOTATEK

```
pair<int, int> para1 = {0, 3};
```



Po znaku **=** w klamrach podajemy **dwie wartości**, oddzielone przecinkiem. Pierwsza trafia do **first**, druga do **second** — po kolei.

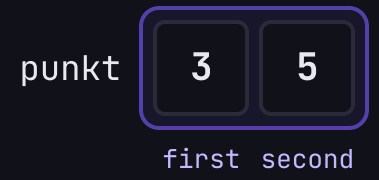
> Wygląda znajomo? To jak lista inicjalizacyjna z tablic — tylko że w parze zawsze są **dokładnie dwie** wartości.

02 Więcej przykładów tworzenia

Każda linijka tworzy jedną parę. Po prawej — co jest w środku:

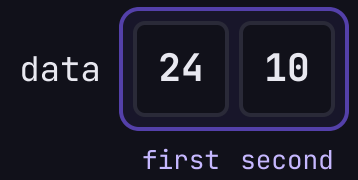
```
pair<int, int> punkt = {3, 5};
```

punkt (x, y)



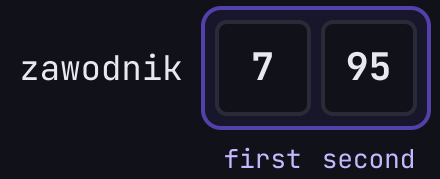

```
pair<int, int> data = {24, 10};
```

dzień i miesiąc




```
pair<int, int> zawodnik = {7, 95};
```

numer i punkty



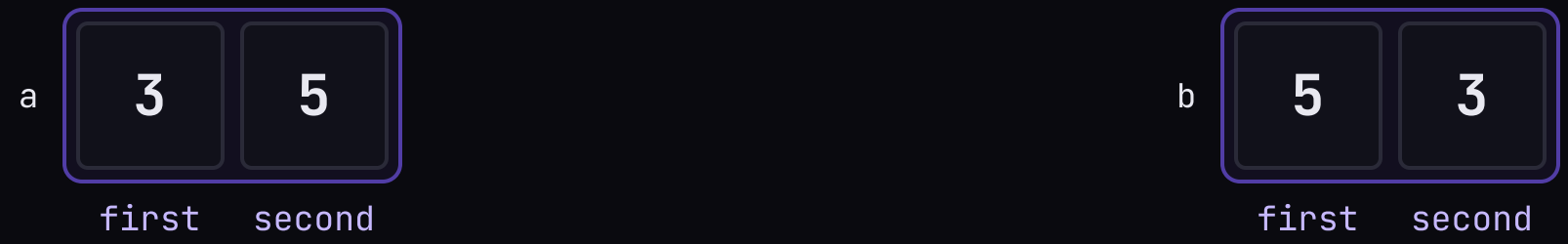

```
pair<int, int> minus = {-4, 0};
```

liczby ujemne i zero też są OK



02 Kolejność ma znaczenie

```
pair<int, int> a = {3, 5};
pair<int, int> b = {5, 3};
```



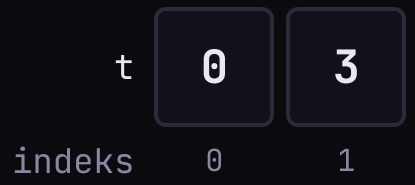
To są **dwie różne pary**. Pierwsza liczba w klamrach zawsze trafia do **first**, druga — do **second**.

Uwaga: punkt (3, 5) to nie to samo co punkt (5, 3)!

02 Para a tablica z dwoma elementami

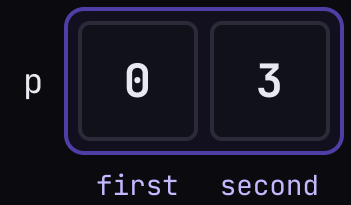
TABLICA — LEKCJA 7

```
int t[2] = {0, 3};
```



PARA — DZISIAJ

```
pair<int, int> p = {0, 3};
```



	Tablica `int t[2]`	Para `pair<int, int> p`
dostęp	`t[0]`, `t[1]` — numery	`p.first`, `p.second` — nazwy
ile elementów	tyle, ile rozmiar	zawsze **2**
typy	wszystkie takie same	mogą być **różne** (rozdział 08)

03 Para bez wartości — co jest w środku?

```
STREFA GLOBALNA — POZA MAIN
#include <iostream>
#include <utility>
using namespace std;

pair<int, int> para1;     // globalna

int main(){
    STREFA LOKALNA — W KLAMRACH MAIN
    pair<int, int> para2;  // lokalna
}
```

Tak jak przy tablicach, parę można stworzyć **globalnie** (poza main) albo **lokalnie** (w main). Nie podajemy żadnych wartości.

Pamiętasz tablice? Tablica bez listy miała **zera** globalnie, a **śmieci** lokalnie.

A para?

03 Przypadek nr 1: para globalna

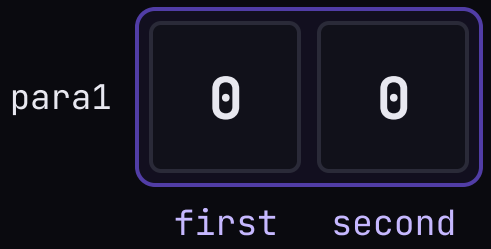
```cpp
#include <iostream>
#include <utility>
using namespace std;

pair<int, int> para1;   // globalna

int main(){

}
```

Para stworzona **poza main** (poza funkcjami i klamrami) ma w środku same zera:



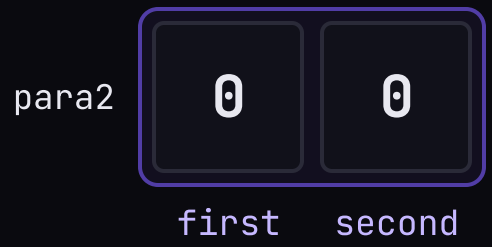
Zapamiętaj: globalna para bez wartości = **{0, 0}**. Zupełnie jak globalna tablica.

03 Przypadek nr 2: para lokalna

```cpp
#include <iostream>
#include <utility>
using namespace std;

int main(){
    pair<int, int> para2;   // lokalna
}
```

Para stworzona **w main** też ma w środku… **zera**!

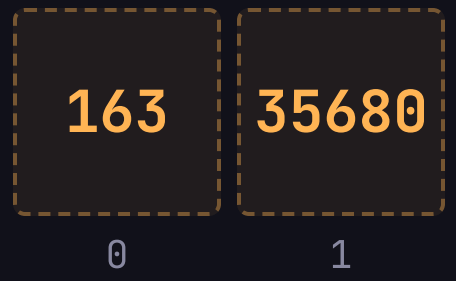


Dlaczego? para ma wbudowany mechanizm, który przy tworzeniu **zawsze zeruje** first i second. Dlatego nawet lokalnie nie ma w niej śmieci.

03 Para a tablica int — porównanie

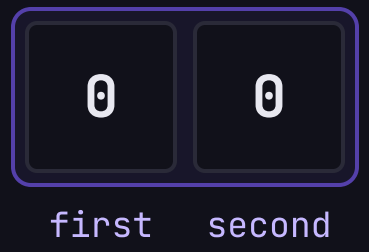
```
int main(){
    int t[2];              // lokalna tablica int → śmieci
    pair<int, int> p;      // lokalna para        → zera
}
```

t — lokalna tablica int



losowe śmieci — np. 163 i 35680

p — lokalna para



zawsze {0, 0}

Dobry nawyk i tak: nadawaj parze wartości od razu, np. `pair<int, int> p = {0, 0};` — od razu widać, co w niej jest.

04 Jak dostać się do jednej przegródki?

```
nazwa.first    nazwa.second
```

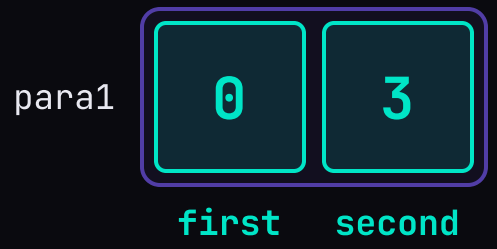
Piszemy nazwę pary, potem **kropkę**, a potem `first` albo `second`. Kropkę czytamy jak „w środku”: `para1.first` = „first w środku para1”.

```
para1.first    // pierwszy element pary
```

```
para1.second   // drugi element pary
```

04 para1.first i para1.second

```
pair<int, int> para1 = {0, 3};
```



Zapis	Co oznacza	Wartość
`para1.first`	przechowuje pierwszy element pary	0
`para1.second`	przechowuje drugi element pary	3

04 Wypisujemy first

```cpp
pair<int, int> para1 = {0, 3};
cout << para1.first << endl;
```



1. C++ bierze parę `para1`.
2. Zagląda do przegródki **first**.
3. Wyjmuje to, co tam leży: **0**.
4. `cout` wypisuje 0, a `endl` przechodzi do nowej linii.

NA EKRANIE

```
0
```

04 Wypisujemy second

```cpp
pair<int, int> para1 = {0, 3};
cout << para1.second << endl;
```



1. C++ bierze parę `para1`.
2. Zagląda do przegródki **second**.
3. Wyjmuje to, co tam leży: **3**.
4. `cout` wypisuje 3.

NA EKRANIE

```
3
```

04 first i second działają jak zwykłe zmienne

`punkt.first` to po prostu liczba int. Można jej użyć wszędzie tam, gdzie zwykłej zmiennej: przypisać, dodać, pomnożyć, wypisać.

```
pair<int, int> punkt = {3, 5};

int x = punkt.first;                          // x = 3
cout << x << endl;
cout << punkt.first + punkt.second << endl;   // 3 + 5
cout << punkt.second * 10 << endl;            // 5 * 10
```

NA EKRANIE

```
3
8
50
```



04 Para nie ma indeksów

ŹLE — PARA TO NIE TABLICA

```
cout << para1[0];   // BŁĄD
```

DOBRZE

```
cout << para1.first;
```

W tablicy szufladki mają **numery**: t[0], t[1]. W parze przegródki mają **nazwy**: first i second. Nawiasów kwadratowych przy parze **nie używamy**.



Uwaga: nie ma też `.third` — para ma **tylko dwa** elementy: first i second.

05 Wypisujemy całą parę — kod z notatek

```
cout << para1.first << " " << para1.second;
```



1. `cout << para1.first` — wypisuje **0**.
2. `<< " "` — wypisuje **spację**, żeby liczby się nie skleiły.
3. `<< para1.second` — wypisuje **3**.

NA EKRANIE

```
0 3
```

05 Nie da się wypisać pary jednym cout

TAK NIE ROBIMY

```
pair<int, int> para1 = {0, 3};
cout << para1 << endl;   // BŁĄD
```

KOMPILATOR MÓWI

```
error: no match for 'operator<<'
```

Przy tablicy pojawiał się dziwny adres. Przy parze jest jeszcze prościej: program **w ogóle się nie skompiluje**. cout nie wie, jak wypisać całą parę.

Nie da się wypisać pary jednym cout.

Zasada: wypisujemy **first** i **second** osobno, a pomiędzy nimi spację.

05 Różne sposoby wypisania tej samej pary

PARA, KTÓRĄ WYPISUJEMY

```
pair<int, int> p = {3, 5};
```

Kod	Na ekranie
`cout << p.first << " " << p.second << endl;`	`3 5`
`cout << "(" << p.first << ", " << p.second << ")" << endl;`	`(3, 5)`
`cout << p.second << " " << p.first << endl;`	`5 3` — odwrotnie
`cout << p.first << endl << p.second << endl;`	3 i 5 w dwóch liniach

Spację można zapisać w cudzysłowie `" "` albo w apostrofach `' '` — oba działają tak samo.

05 Cały program

```cpp
#include <iostream>
#include <utility>
using namespace std;

int main(){
    pair<int, int> para1 = {0, 3};
    cout << para1.first << " " << para1.second << endl;
}
```

NA EKRANIE

```
0 3
```

Para stoi w main, więc jest **lokalna** — ale ma wartości od razu, więc w środku jest dokładnie {0, 3}.

06 Wpisujemy nową wartość do first

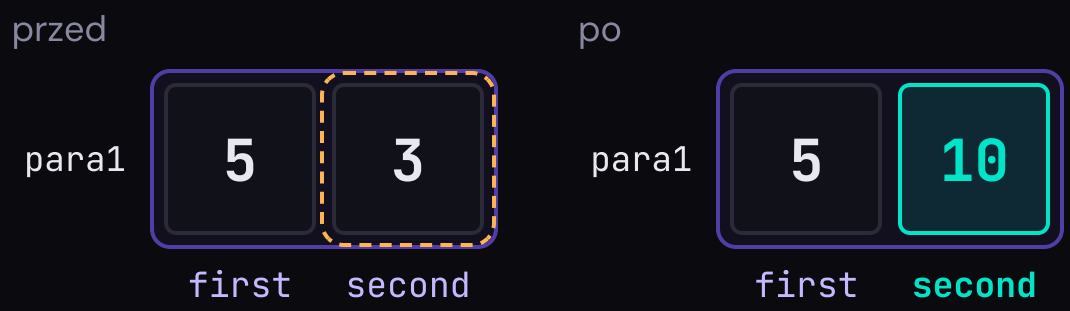
```cpp
pair<int, int> para1 = {0, 3};
para1.first = 5;
```



Zmieniamy **tylko first**: stare 0 znika, w jego miejscu jest 5. second zostaje bez zmian (3).

06 Tak samo z second

```
// para1 = {5, 3}
para1.second = 10;
```



Teraz zmieniamy **tylko second**: 3 → 10. first zostaje bez zmian (5).

06 Przypomnienie: skróty ++, *=, %=

Skrót	To samo co	Co robi
`x++`	`x = x + 1`	zwiększa o 1
`x += 4`	`x = x + 4`	dodaje 4
`x -= 4`	`x = x - 4`	odejmuje 4
`x *= 20`	`x = x * 20`	mnoży przez 20
`x %= 10`	`x = x % 10`	zostawia resztę z dzielenia przez 10

Ważne: te skróty działają z `para1.first` i `para1.second` dokładnie tak samo jak ze zwykłą zmienną.

06 para1.first++

```
// para1 = {5, 10}
para1.first++;
```



first było **5**. Dodajemy 1 → **6**. second się nie zmienia.

06 para1.second *= 20

```
// para1 = {6, 10}
para1.second *= 20;
```



second było **10**. Mnożymy przez 20 → 10 · 20 = **200**.

06 para1.first %= 10

```
// para1 = {6, 200}
para1.first %= 10;
```



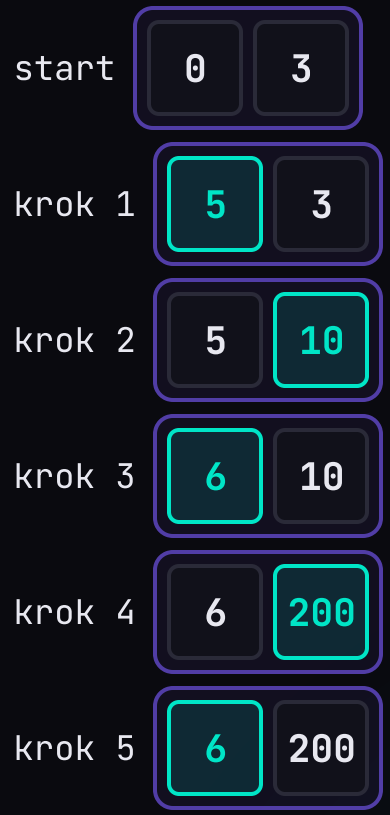
first było **6**. Reszta z dzielenia 6 przez 10 to **6** — więc wartość się nie zmienia. Gdyby first było 26, zostałoby 6.

06 Wszystkie zmiany z notatek po kolei

```cpp
pair<int, int> para1 = {0, 3};   // start
para1.first = 5;                 // krok 1
para1.second = 10;               // krok 2
para1.first++;                   // krok 3
para1.second *= 20;              // krok 4
para1.first %= 10;               // krok 5
cout << para1.first << " " << para1.second;
```

NA EKRANIE

```
6 200
```



07 Nowe wartości dla obu naraz

Z NOTATEK

```
// para1 = {6, 200}
para1 = {3, 7};
```



DWIE LINIJKI…

```
para1.first = 3;
para1.second = 7;
```

…ALBO JEDNA — TO SAMO

```
para1 = {3, 7};
```

Gdy zmieniamy **oba** elementy, przypisujemy całą parę naraz — w klamrach, jak przy tworzeniu.

07 Nowa para zbudowana ze starej

Z NOTATEK

```
// para1 = {3, 7}
para1 = {para1.first + 1, para1.second * 2};
```

1. C++ najpierw liczy to, co w klamrach — używając **starych** wartości: 3 + 1 = **4** oraz 7 · 2 = **14**.
2. Dopiero potem wkłada obie nowe liczby do pary: first = 4, second = 14.



NA EKRANIE PO COUT << PARA1.FIRST << " " << PARA1.SECOND

```
4 14
```

07 Jeden element czy dwa naraz?

Modyfikowanie jednego

```
para1.first = 5;
para1.second = 10;
para1.first++;
para1.second *= 20;
para1.first %= 10;
```

Zmienia się **tylko** ta przegródka, którą wskazujemy po kropce.

Modyfikowanie dwóch

```
para1 = {3, 7};
para1 = {para1.first + 1, para1.second * 2};
```

Nowe wartości dla **first i second** naraz — w klamrach, po kolei.

08 W nawiasach ostrych wybieramy typy

Typy w parze nie muszą być takie same. Przykład z notatek:

```
pair<char, int> para2 = {'c', 123};
     ①     ②        ③     ④
```

①	`char`	typ **first** — jeden znak
②	`int`	typ **second** — liczba całkowita
③	`'c'`	wartość first — znak w apostrofach
④	`123`	wartość second

W nawiasach ostrych < > piszemy, **jakie typy przechowuje nasza para**. Pierwszy typ jest dla first, drugi — dla second.

08 para2 krok po kroku

```
pair<char, int> para2 = {'c', 123};
cout << para2.first << endl;    // c
cout << para2.second << endl;   // 123
```

NA EKRANIE

```
c
123
```



first jest typu **char**, więc trzyma literę. second jest typu **int**, więc trzyma liczbę.

08 Więcej par z różnymi typami

```
pair<string, int> uczen = {"Ala", 95};
```

imię i punkty




```
pair<int, double> pomiar = {3, 36.6};
```

dzień i temperatura




```
pair<char, char> znaki = {'x', 'y'};
```

dwa znaki



string znasz z poprzedniej lekcji. Gdy używasz go w parze, dopisz na górze `#include <string>`.

08 Kolejność typów = kolejność wartości

ŹLE

```
pair<char, int> p = {123, 'c'};   // odwrotnie!
cout << p.first << " " << p.second;
```

NA EKRANIE

```
{ 99
```

Program się skompiluje, ale wynik jest dziwny: liczba 123 wpadła do **char** — C++ zamienił ją na znak o numerze 123, czyli **{**. A znak 'c' wpadł do **int** — jako swój numer, czyli **99**.



Zawsze: pierwsza wartość pasuje do pierwszego typu, druga — do drugiego: `{'c', 123}`.

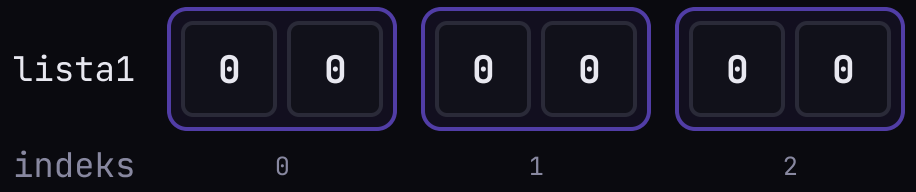
09 W tablicy mogą leżeć pary

Na lekcji o tablicach w szufladkach leżały liczby. Ale w szufladce może leżeć też **para**!

```
pair<int, int> lista1[3];
```

1 2 3 4

1	`pair<int, int>`	typ **każdej szufladki** — para dwóch intów
2	`lista1`	**nazwa** tablicy
3	`[3]`	**rozmiar** — 3 szufladki, w każdej jedna para
4	`;`	**średnik**



09 Tablica par od razu z wartościami

WARTOŚCI Z NOTATEK

```
pair<int, int> lista2[3] = {{0, 0}, {1, 5}, {-1, 7}};
```



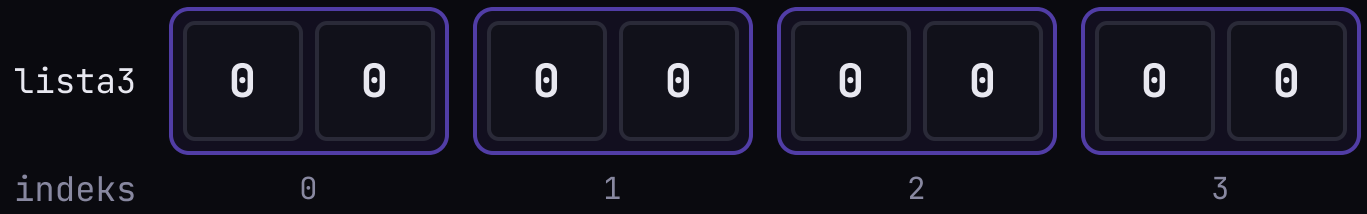
Klamry w klamrach: zewnętrzne klamry to cała tablica, wewnętrzne (w ramkach) — kolejne pary. Każda para trafia do następnej szufladki.

> Rozmiar można pominąć, tak jak w tablicy int: `pair<int, int> lista2[] = {{0, 0}, {1, 5}, {-1, 7}};` — C++ policzy 3 pary.

09 Tablica par bez wartości

Z NOTATEK (TAM Z VECTOREM — ZOBACZYSZ GO NA NASTĘPNEJ LEKCJI)

```
pair<int, int> lista3[4];
```



Każda szufladka to para, a każda para przy tworzeniu **sama się zeruje**. Dlatego w tablicy par bez wartości są same **{0, 0}** — i globalnie, **i lokalnie**.

> Inaczej niż w lokalnej tablicy int, gdzie były śmieci!

09 Najpierw indeks, potem kropka

```
lista2[1].first
```

1. `lista2` — bierzemy tablicę.
2. `[1]` — idziemy do szufladki o indeksie 1. Leży tam **para {1, 5}**.
3. `.first` — z tej pary bierzemy first → **1**.



Uwaga: kolejność jest ważna: `lista2[1].first`, a nie `lista2.first[1]`. Najpierw wybieramy szufladkę, potem przegródkę.

09 Odczyt z tablicy par — przykłady

```
pair<int, int> lista2[3] = {{0, 0}, {1, 5}, {-1, 7}};

cout << lista2[1].first << endl;    // 1
cout << lista2[1].second << endl;   // 5
cout << lista2[2].first << endl;    // -1
cout << lista2[2].second << endl;   // 7
```

NA EKRANIE

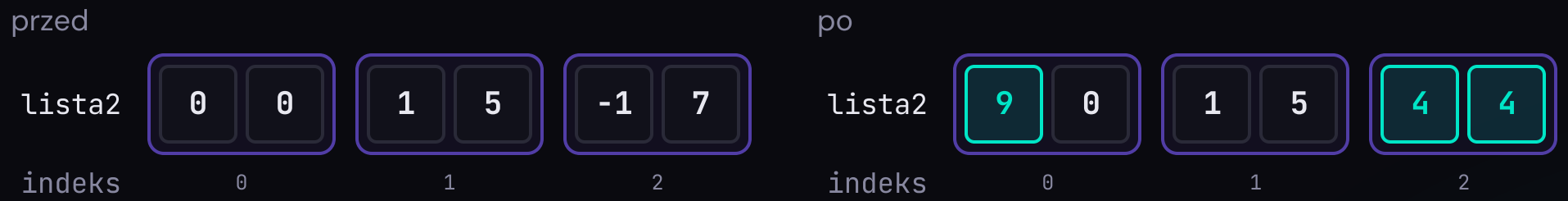
```
1
5
-1
7
```



`lista2[1]` to cała para {1, 5}. Dopiero `.first` albo `.second` wybiera z niej jedną liczbę.

09 Zmiana w tablicy par

```
pair<int, int> lista2[3] = {{0, 0}, {1, 5}, {-1, 7}};
lista2[0].first = 9;    // zmienia JEDNĄ liczbę w parze
lista2[2] = {4, 4};     // zmienia CAŁĄ parę
```

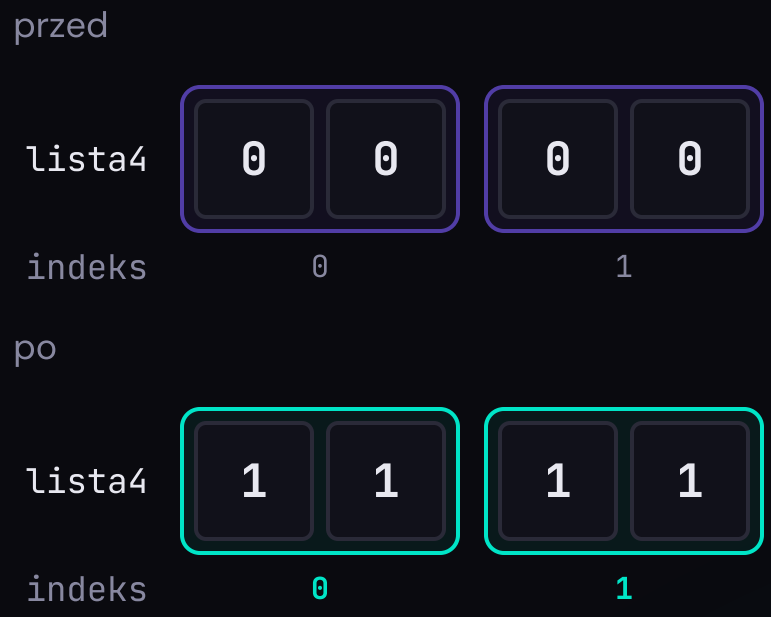


To te same dwa sposoby co przy pojedynczej parze: przez kropkę zmieniamy **jeden** element, przez klamry — **oba**.

09 Wypełnianie tablicy par pętlą

```
pair<int, int> lista4[2];
for(int i = 0; i < 2; i++){
    lista4[i] = {1, 1};
}
```

Tak jak na lekcji o tablicach: pętla przechodzi po indeksach i do **każdej** szufladki wpisuje parę {1, 1}.

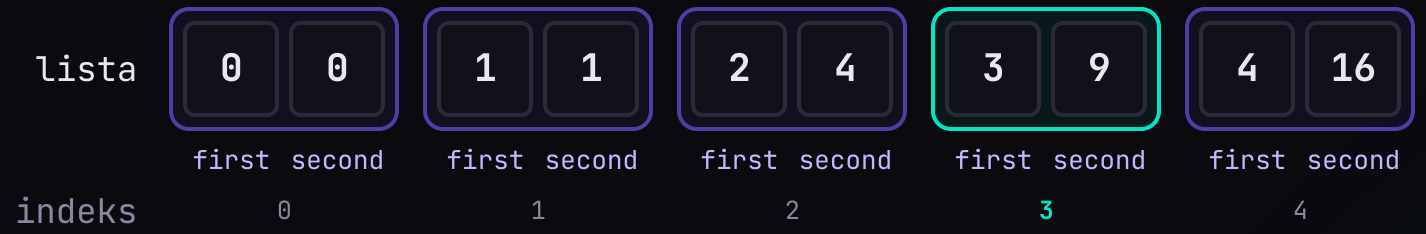


W notatkach ten przykład był z vectorem: lista4(2, {1, 1}) — dwie pary {1, 1}.

09 Pary zależne od i

```
pair<int, int> lista[5];
for(int i = 0; i < 5; i++){
    lista[i] = {i, i * i};
}
```

W każdym obrocie robimy parę z dwóch liczb: **i** oraz **i · i**. Dla i = 3 to para {3, 9}.



09 Wypisywanie całej tablicy par

```
pair<int, int> lista2[3] =
    {{0, 0}, {1, 5}, {-1, 7}};
for(int i = 0; i < 3; i++){
    cout << lista2[i].first << " "
         << lista2[i].second << endl;
}
```

Jedna linijka wyjścia = jedna para. Dla czytelności polecenie cout jest rozbite na **dwie linijki** — to wciąż jedno polecenie, kończy się dopiero na średniku.

NA EKRANIE

```
0 0
1 5
-1 7
```

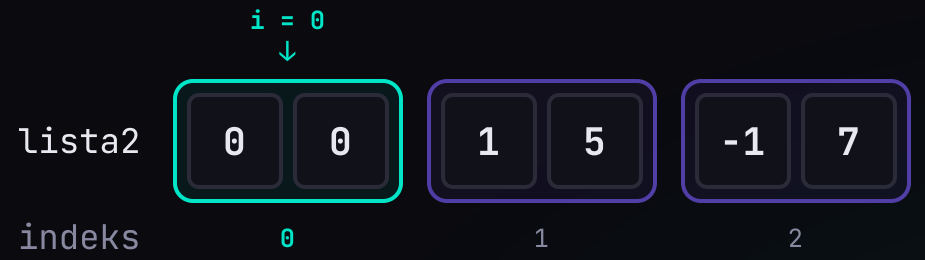
Pętla idzie po indeksach 0, 1, 2 — jak przy tablicy int. W środku wypisujemy first, spację, second i endl.

09 Obrót 1 z 3: i = 0

```
pair<int, int> lista2[3] =
    {{0, 0}, {1, 5}, {-1, 7}};
for(int i = 0; i < 3; i++){
    cout << lista2[i].first << " "
         << lista2[i].second << endl;
}
```

NA EKRANIE

i	0
warunek	`0 < 3` → TAK
wypisujemy	`lista2[0].first` = 0, spację, `lista2[0].second` = 0, nową linię



09 Obrót 2 z 3: i = 1

```
pair<int, int> lista2[3] =
    {{0, 0}, {1, 5}, {-1, 7}};
for(int i = 0; i < 3; i++){
    cout << lista2[i].first << " "
         << lista2[i].second << endl;
}
```

NA EKRANIE

```
0 0
```

i	1
warunek	`1 < 3` → TAK
wypisujemy	`lista2[1].first` = 1, spację, `lista2[1].second` = 5, nową linię



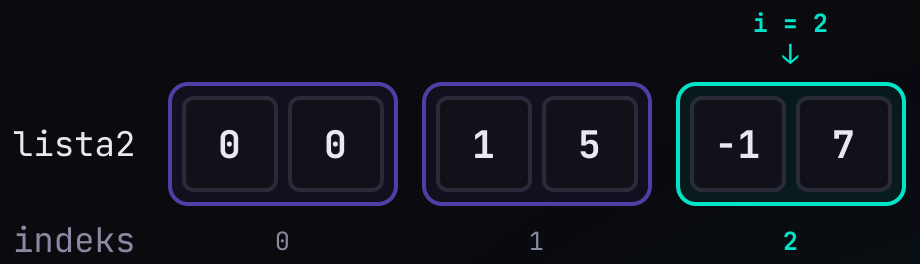
09 Obrót 3 z 3: i = 2

```
pair<int, int> lista2[3] =
    {{0, 0}, {1, 5}, {-1, 7}};
for(int i = 0; i < 3; i++){
    cout << lista2[i].first << " "
         << lista2[i].second << endl;
}
```

NA EKRANIE

```
0 0
1 5
```

i	2
warunek	`2 < 3` → TAK
wypisujemy	`lista2[2].first` = -1, spację, `lista2[2].second` = 7, nową linię



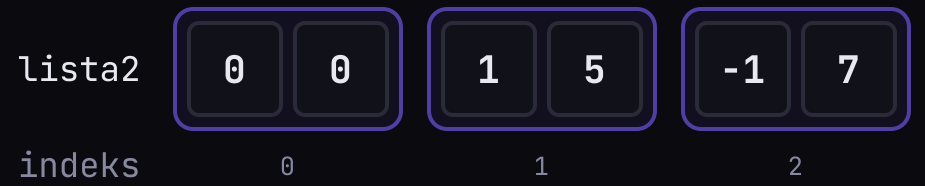
09 i = 3: koniec pętli

```
pair<int, int> lista2[3] =
    {{0, 0}, {1, 5}, {-1, 7}};
for(int i = 0; i < 3; i++){
    cout << lista2[i].first << " "
         << lista2[i].second << endl;
}
```

NA EKRANIE

```
0 0
1 5
-1 7
```

i	3
warunek	3 < 3 → NIE — koniec pętli



Gotowe: wypisaliśmy wszystkie 3 pary, każdą w osobnej linii.

10 Para pomocnicza tmp

```
pair<int, int> lista[10];
pair<int, int> tmp = {3, 5};
lista[0] = tmp;
```

1. Tablica na 10 par — na razie same {0, 0}.
2. Robimy parę pomocniczą `tmp` = {3, 5}.
3. `lista[0] = tmp;` — **kopia** tmp ląduje w szufladce 0.

Czasem najpierw robimy parę **pomocniczą** (tymczasową), a dopiero potem wkładamy ją do tablicy. Taką parę często nazywa się **tmp** — od angielskiego „temporary”, czyli tymczasowy.



10 make_pair — para bez nazwy

Z PARĄ TMP

```
pair<int, int> tmp = {3, 5};
lista[0] = tmp;
```

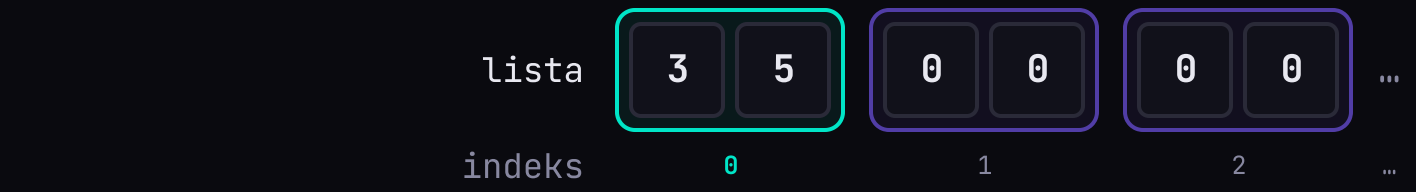
=

Z MAKE_PAIR

```
lista[0] = make_pair(3, 5);
```

To jest **to samo**! Zamiast robić osobną parę tmp, piszemy od razu `make_pair(3, 5)`.

> make_pair tworzy parę „w locie” — **bez nazwy** i bez osobnej linijki. Taka para żyje tylko chwilę: dokładnie tyle, żeby ją gdzieś wpisać. Dlatego nazywamy ją **tymczasową**.



10 Budowa make_pair

```
make_pair(3, 5)
```
1 2 3

1	`make_pair`	„zrób parę”
2	`3`	pierwsza wartość — trafi do **first**
3	`5`	druga wartość — trafi do **second**

Typy C++ **zgaduje sam** z podanych wartości: `make_pair(3, 5)` to para dwóch intów, a `make_pair('c', 123)` to para char i int.

10 Trzy sposoby — ten sam efekt

Sposób	Kod	Wynik
z parą tmp	`pair<int, int> tmp = {3, 5};` `lista[0] = tmp;`	`lista[0] = {3, 5}`
make_pair	`lista[0] = make_pair(3, 5);`	`lista[0] = {3, 5}`
klamry	`lista[0] = {3, 5};`	`lista[0] = {3, 5}`

Wszystkie trzy dają dokładnie to samo. make_pair przydaje się szczególnie, gdy wartości mamy w zmiennych — np. zaraz po wczytaniu z klawiatury (następny rozdział).

10 make_pair ze zmiennymi

```
int a = 3;
int b = 5;
pair<int, int> lista[10];
lista[0] = make_pair(a, b);          // {3, 5}
lista[1] = make_pair(b, a);          // {5, 3}
lista[2] = make_pair(a + b, a * b);  // {8, 15}
```



Do make_pair można wstawić zmienne, a nawet całe działania — C++ najpierw je policzy.

11 Jak wygląda wejście?

FORMAT WEJŚCIA

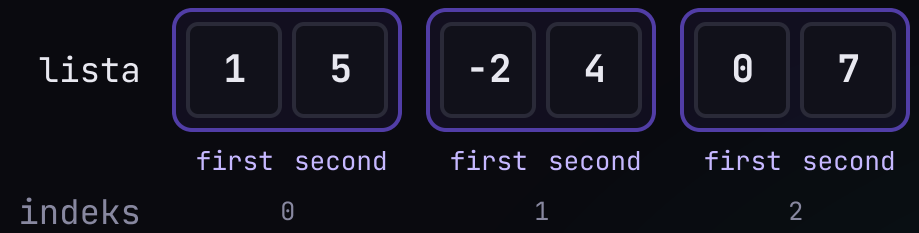
n
x_0 y_0
x_1 y_1
…
x_{n-1} y_{n-1}

W pierwszej linii **n** — ile par. Potem **n linijek**, w każdej **dwie liczby**, czyli jedna para.

PRZYKŁAD

```
3
1 5
-2 4
0 7
```

Tutaj n = 3, więc są 3 linijki z parami.



11 Sposób 1: wczytujemy prosto do first i second

```
pair<int, int> lista[100]; // 100 >= n
int n;
cin >> n;
for(int i = 0; i < n; i++){
    cin >> lista[i].first >> lista[i].second;
}
```

W każdym obrocie pętli `cin` czyta **dwie** liczby: pierwszą wkłada do `lista[i].first`, drugą do `lista[i].second`.

> Tablica jest z zapasem — dokładnie jak na lekcji o tablicach. Pętla idzie do **n**, a nie do 100.

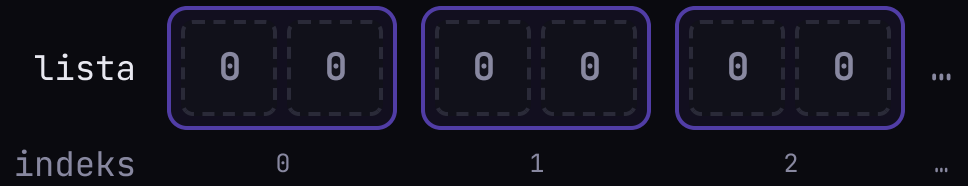
11 Wczytujemy przykład — start: n = 3

```
pair<int, int> lista[100]; // 100 >= n
int n;
cin >> n;
for(int i = 0; i < n; i++){
    cin >> lista[i].first >> lista[i].second;
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
3
1 5
-2 4
0 7
```

robimy	`cin >> n;`
n	3



Szare zera to pary, do których jeszcze nic nie wczytaliśmy (para sama się zeruje).

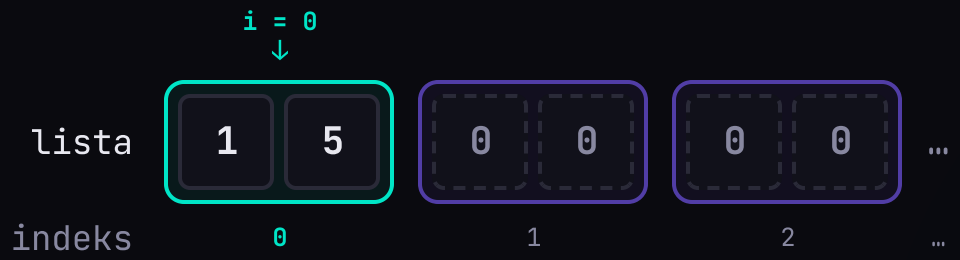
11 Obrót 1 z 3: i = 0

```
pair<int, int> lista[100]; // 100 >= n
int n;
cin >> n;
for(int i = 0; i < n; i++){
    cin >> lista[i].first >> lista[i].second;
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
3
1 5
-2 4
0 7
```

i	0
warunek	`0 < 3` → TAK
wczytane	1 → `lista[0].first`, 5 → `lista[0].second`



11 Obrót 2 z 3: i = 1

```
pair<int, int> lista[100]; // 100 >= n
int n;
cin >> n;
for(int i = 0; i < n; i++){
    cin >> lista[i].first >> lista[i].second;
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
3
1 5
-2 4
0 7
```

i	1
warunek	1 < 3 → TAK
wczytane	-2 → lista[1].first, 4 → lista[1].second



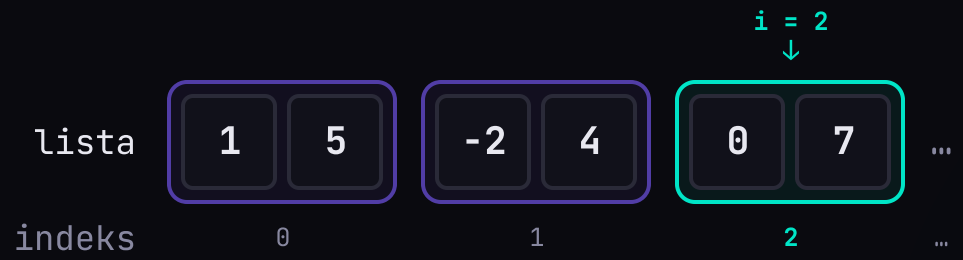
11 Obrót 3 z 3: i = 2

```
pair<int, int> lista[100]; // 100 >= n
int n;
cin >> n;
for(int i = 0; i < n; i++){
    cin >> lista[i].first >> lista[i].second;
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
3
1 5
-2 4
0 7
```

i	2
warunek	`2 < 3` → TAK
wczytane	0 → `lista[2].first`, 7 → `lista[2].second`



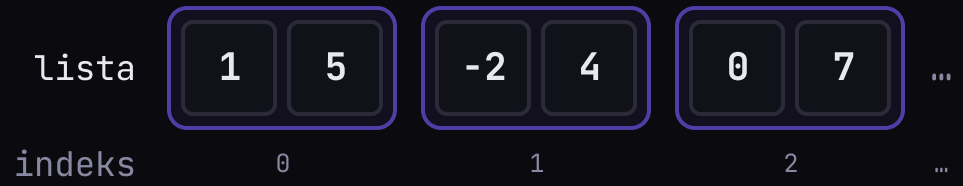
11 Koniec wczytywania: i = 3

```
pair<int, int> lista[100]; // 100 >= n
int n;
cin >> n;
for(int i = 0; i < n; i++){
    cin >> lista[i].first >> lista[i].second;
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
3
1 5
-2 4
0 7
```

i	3
warunek	3 < 3 → NIE



Gotowe: wszystkie 3 pary leżą w szufladkach 0, 1, 2 — czyli **0…n−1**.

11 Sposób 2: zmienne pomocnicze i `make_pair`

```
for(int i = 0; i < n; i++){
    int a, b;
    cin >> a >> b;
    lista[i] = make_pair(a, b);
}
```

1. Tworzymy dwie zwykłe zmienne **a** i **b**.
2. Wczytujemy do nich dwie liczby z linijki.
3. Robimy z nich tymczasową parę `make_pair(a, b)` i wkładamy ją do szufladki i.

Efekt jest taki sam jak w sposobie 1. Zamiast make_pair można też napisać `{a, b}`.

11 Wczytaj i wypisz — cały program

```cpp
#include <iostream>
#include <utility>
using namespace std;

int main(){
    pair<int, int> lista[100]; // 100 >= n
    int n;
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> lista[i].first >> lista[i].second;
    }
    for(int i = 0; i < n; i++){
        cout << lista[i].first << " "
             << lista[i].second << endl;
    }
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
3
1 5
-2 4
0 7
```

NA EKRANIE

```
1 5
-2 4
0 7
```

11 Ten sam program — inne wejścia

Nie zmieniamy **ani jednej linijki** kodu — program sam dopasowuje się do n:

WEJŚCIE

```
1
10 20
```

NA EKRANIE

```
10 20
```

WEJŚCIE

```
2
4 4
-1 0
```

NA EKRANIE

```
4 4
-1 0
```

WEJŚCIE

```
4
1 2
3 4
5 6
7 8
```

NA EKRANIE

```
1 2
3 4
5 6
7 8
```

Działa dla każdego n od 1 do 100 — bo tyle par mieści tablica.

12 W parze może leżeć… para

W nawiasach ostrych może stać **dowolny typ** — także inna para! Wtedy first jest całą parą.

```
pair<pair<int, int>, int> p = {{1, 2}, 3};
```



Przykład z życia: punkt (x, y) i jego numer → {{x, y}, numer}. W notatkach w parze leżały jeszcze inne struktury — poznasz je na kolejnych lekcjach.

12 Jak dostać się do środka?

Zapis	Co to jest	Wartość
`p.first`	cała wewnętrzna para	{1, 2}
`p.first.first`	first wewnętrznej pary	1
`p.first.second`	second wewnętrznej pary	2
`p.second`	drugi element zewnętrznej pary	3

```
cout << p.first.first << " " << p.first.second << " " << p.second;
```

NA EKRANIE

```
1 2 3
```

Czytamy od lewej: p → .first (wchodzimy do wewnętrznej pary) → .first (bierzemy jej pierwszy element).

13 Pytanie 1: co wypisze program?

```
pair<int, int> p = {4, 9};
cout << p.second << " " << p.first << endl;
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 1: 9 4



1. Najpierw wypisujemy `p.second` → **9**.
2. Potem spację.
3. Na końcu `p.first` → **4**. Kolejność w cout decyduje o kolejności na ekranie.

NA EKRANIE

```
9 4
```

13 Pytanie 2: para bez wartości

```cpp
int main(){
    pair<int, int> p;
    cout << p.first << " " << p.second;
}
```

NA EKRANIE

```
?
```

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 2: 0 0



Para jest lokalna (w main), ale para **zawsze sama się zeruje** — nie ma w niej śmieci, inaczej niż w lokalnej tablicy int.

NA EKRANIE

```
0 0
```

13 Pytanie 3: zmiany po kolei

```
pair<int, int> p = {2, 5};
p.first++;
p.second *= 3;
p.first %= 2;
cout << p.first << " " << p.second;
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 3: 1 15

Linijka	first	second
`p = {2, 5}`	2	5
`p.first++`	3	5
`p.second *= 3`	3	15
`p.first %= 2`	1	15



NA EKRANIE

```
1 15
```

3 % 2 = 1 — reszta z dzielenia 3 przez 2.

13 Pytanie 4: zamiana miejscami

```
pair<int, int> p = {3, 7};
p = {p.second, p.first};
cout << p.first << " " << p.second;
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 4: 7 3



1. C++ najpierw liczy klamry ze **starymi** wartościami: {p.second, p.first} = {7, 3}.
2. Potem wkłada tę nową parę do p. Elementy zamieniły się miejscami!

NA EKRANIE

```
7 3
```

13 Pytanie 5: tablica par

```
pair<int, int> t[3] = {{1, 2}, {3, 4}, {5, 6}};
cout << t[1].first + t[2].second << endl;
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 5: 9



1. `t[1]` to para {3, 4}, więc `t[1].first` = **3**.
2. `t[2]` to para {5, 6}, więc `t[2].second` = **6**.
3. 3 + 6 = **9**.

NA EKRANIE

```
9
```

13 Pytanie 6: znajdź dwa błędy

```
pair<int, int> p = {1, 2};
cout << p << endl;
cout << p[0] << endl;
```

Program miał wypisać obie liczby z pary. Co jest nie tak?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 6: cout i nawiasy

ŹLE

```
cout << p << endl;
cout << p[0] << endl;
```

DOBRZE

```
cout << p.first << " " << p.second << endl;
```

01 `cout << p` — nie da się wypisać pary jednym cout (błąd kompilacji).

02 `p[0]` — para nie ma indeksów. Pierwszy element to `p.first`.

14 Najczęstsze błędy (1/2)

01 `cout << para;` — nie da się wypisać pary jednym cout (błąd kompilacji). Wypisz first i second osobno.

02 `para[0]`, `para[1]` — para to nie tablica, nie ma indeksów. Używamy `.first` i `.second`.

03 **Trzy wartości.** `pair<int, int> p = {1, 2, 3};` — błąd. Para ma zawsze dokładnie dwa elementy; nie ma też `.third`.

04 **Zła kolejność wartości.** `pair<char, int> p = {123, 'c'};` — first to „{”, second to 99. Kolejność wartości = kolejność typów.

14 Najczęstsze błędy (2/2)

01 **Brak kropki albo wielka litera.** `para1first`, `para1.First` — C++ rozróżnia wielkość liter. Poprawnie: `para1.first`.

02 **Kolejność w tablicy par.** `lista.first[1]` zamiast `lista[1].first` — najpierw indeks w nawiasach, potem kropka.

03 **Złe nawiasy przy typach.** `pair(int, int)` albo `pair[int, int]` — typy pary piszemy w nawiasach ostrych: `pair<int, int>`.

04 **Zmiana obu bez klamer.** `para1 = 3, 7;` — błąd. Poprawnie: `para1 = {3, 7};` albo `para1 = make_pair(3, 7);`.

15 Ściąga: para

Tworzenie z wartościami

`pair<int, int> p = {0, 3};`

Bez wartości

`pair<int, int> p;` → {0, 0} globalnie i lokalnie

Pierwszy / drugi element

`p.first` · `p.second`

Wypisanie

`cout << p.first << " " << p.second;`

Zmiana jednego

`p.first = 5;` · `p.second *= 20;`

Zmiana obu

`p = {3, 7};`

Różne typy

`pair<char, int> p = {'c', 123};`

Para tymczasowa

`make_pair(3, 5)`

Tablica par

`pair<int, int> t[3];` → `t[1].first`

Cała para jednym cout?

NIE — first i second osobno

15 Ściąga: tablica par

TABLICA PAR Z WARTOŚCIAMI

```
pair<int, int> t[3] = {{0, 0}, {1, 5}, {-1, 7}};
```

WYPEŁNIJ PĘTLĄ

```
for(int i = 0; i < n; i++){
    t[i] = make_pair(i, i * i);
}
```

WCZYTAJ N PAR

```
pair<int, int> t[100]; // 100 >= n
int n;
cin >> n;
for(int i = 0; i < n; i++){
    cin >> t[i].first >> t[i].second;
}
```

WYPISZ N PAR

```
for(int i = 0; i < n; i++){
    cout << t[i].first << " "
         << t[i].second << endl;
}
```

15 Zadania na koniec — Twoja kolej

1. Stwórz parę {12, 30} (godzina i minuta) i wypisz ją w formacie 12:30.
2. Stwórz parę {4, 9}, zamień jej elementy miejscami jedną linijką i wypisz ją.
3. Wczytaj dwie liczby, zrób z nich parę przez make_pair i wypisz najpierw second, potem first.
4. Wczytaj n (n ≤ 100) i n par liczb. Dla każdej pary wypisz sumę first + second — każdą w osobnej linii.
5. Wczytaj n par (x, y) i wypisz tylko te, w których x jest większe od y.

// KONIEC LEKCJI 9

Umiesz już korzystać z par!

Tworzenie, first i second, wypisywanie, zmiana jednego i obu elementów, różne typy, tablica par, make_pair i wczytywanie. Teraz zrób zadania z poprzedniego slajdu. Na następnej lekcji: **vector** — tam pary przydadzą się jeszcze bardziej.

